

Інструкція для студентів

Інструкція з оформлення робіт для TestHub

Загальна інформація

Система TestHub приймає роботи у форматі **PDF**. Рекомендуємо писати роботи в Markdown (.md), а потім конвертувати в PDF.

Інструменти для роботи

Редактори з підтримкою Markdown + формул:

Редактор	Платформа	Особливості
VS Code + розширення	Windows/Mac/Linux	Безкоштовний, потужний
Typora	Windows/Mac/Linux	Платний, дуже зручний
Obsidian	Windows/Mac/Linux	Безкоштовний
StackEdit	Браузер	Онлайн, безкоштовний
HackMD	Браузер	Онлайн, командна робота

Розширення для VS Code:

- **Markdown All in One** — базова підтримка
- **Markdown Preview Enhanced** — попередній перегляд + експорт в PDF
- **Markdown+Math** — підтримка LaTeX формул

Математичні формули (LaTeX)

Формули в рядку тексту

Використовуй одинарні долари `$...$`:

Теорема Піфагора: `$a^2 + b^2 = c^2$`

Корінь рівняння: `$x = \frac{-b \pm \sqrt{b^2-4ac}}{2a}$`

Результат: Теорема Піфагора: $a^2 + b^2 = c^2$

Формули окремим блоком

Використовуй подвійні долари `$$...$$`:

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \frac{\sqrt{\pi}}{2}$$

Основні математичні символи

Арифметика та алгебра

Опис	Код	Результат
Дріб	<code>\frac{a}{b}</code>	a/b
Степінь	<code>x^2</code> або <code>x^{10}</code>	x^2
Індекс	<code>x_1</code> або <code>x_{12}</code>	x_1
Корінь	<code>\sqrt{x}</code>	$\sqrt{x}$
n-й корінь	<code>\sqrt[n]{x}</code>	$\sqrt[n]{x}$
Плюс-мінус	<code>\pm</code>	$\pm$
Множення	<code>\times</code> або <code>\cdot</code>	$\times$ або $\cdot$
Ділення	<code>\div</code>	$\div$
Нерівність	<code>\neq</code>	$\neq$
Приблизно	<code>\approx</code>	$\approx$
Менше-рівне	<code>\leq</code>	$\leq$
Більше-рівне	<code>\geq</code>	$\geq$

Грецькі літери

Код	Результат	Код	Результат
<code>\alpha</code>	α	<code>\Alpha</code>	A
<code>\beta</code>	β	<code>\Beta</code>	B
<code>\gamma</code>	γ	<code>\Gamma</code>	Γ
<code>\delta</code>	δ	<code>\Delta</code>	Δ
<code>\epsilon</code>	ϵ	<code>\pi</code>	π
<code>\theta</code>	θ	<code>\lambda</code>	λ
<code>\sigma</code>	σ	<code>\omega</code>	ω
<code>\phi</code>	ϕ	<code>\Phi</code>	Φ

Тригонометрія

Функція	Код	Функція	Код
sin	<code>\sin x</code>	arcsin	<code>\arcsin x</code>
cos	<code>\cos x</code>	arccos	<code>\arccos x</code>
tan	<code>\tan x</code>	arctan	<code>\arctan x</code>
ctg	<code>\cot x</code>	arcctg	<code>\text{arcctg } x</code>
sec	<code>\sec x</code>	csc	<code>\csc x</code>

Основна тотожність: $\sin^2 x + \cos^2 x = 1$

Формула зведення: $\sin(90^\circ - x) = \cos x$

Подвійний кут: $\sin 2x = 2 \sin x \cos x$

Степені: $\sin^2 x$, $\cos^3 x$, $\tan^{-1} x$

Логарифми

Функція	Код	Опис
$\ln$	<code>\ln x</code>	натуральний логарифм
$\log$	<code>\log x</code>	логарифм
$\log$ з основою	<code>\log_{2} x</code>	логарифм за основою 2
$\lg$	<code>\lg x</code>	десятковий логарифм
$\exp$	<code>\exp(x)</code>	експонента

Логарифм добутку: $\log(ab) = \log a + \log b$

Логарифм частки: $\log \frac{a}{b} = \log a - \log b$

Логарифм степеня: $\log a^n = n \log a$

Перехід до іншої основи: $\log_a b = \frac{\ln b}{\ln a}$

Натуральний логарифм: $\ln e^x = x$

Формула Ейлера: $e^{ix} = \cos x + i \sin x$

Calculus (Аналіз)

Похідна: $f'(x)$ або $\frac{df}{dx}$ або $\frac{d^2f}{dx^2}$

Частинна похідна: $\frac{\partial f}{\partial x}$

Інтеграл: $\int f(x) dx$

Визначений інтеграл: $\int_a^b f(x) dx$

Подвійний інтеграл: $\iint_D f(x,y) dA$

Границя: $\lim_{x \rightarrow \infty} f(x)$

Сума: $\sum_{i=1}^n a_i$

Добуток: $\prod_{i=1}^n a_i$

Матриці

```


$$\begin{pmatrix}
a_{11} & a_{12} & a_{13} \\
a_{21} & a_{22} & a_{23} \\
a_{31} & a_{32} & a_{33}
\end{pmatrix}$$


```

Інші типи дужок: - `pmatrix` — круглі дужки $()$ - `bmatrix` — квадратні дужки $[\]$ - `vmatrix` — визначник $||$
 - `Bmatrix` — фігурні дужки $\{ \}$

Системи рівнянь

```


$$\begin{cases} 2x + 3y = 7 \\ x - y = 1 \end{cases}$$


```

Вектори

Вектор: $\vec{a}$ або $\mathbf{a}$

Скалярний добуток: $\vec{a} \cdot \vec{b}$

Векторний добуток: $\vec{a} \times \vec{b}$

Модуль: $|\vec{a}|$ або $\|\vec{a}\|$

Приклад контрольної з математики

```

# Контрольна робота №1
## Тема: Похідні функцій

```

```

**Студент:** Іванов Іван Іванович
**Група:** КН-21
**Дата:** 06.02.2026

```

Завдання 1

Знайти похідну функції $f(x) = x^3 + 2x^2 - 5x + 1$

Розв'язок:

Застосуємо правило диференціювання:

```


$$f'(x) = \frac{d}{dx}(x^3) + \frac{d}{dx}(2x^2) - \frac{d}{dx}(5x) + \frac{d}{dx}(1)$$


```

```


$$f'(x) = 3x^2 + 4x - 5$$


```

Відповідь: $f'(x) = 3x^2 + 4x - 5$

Завдання 2

Обчислити визначений інтеграл $\int_0^2 (x^2 + 1) dx$

Розв'язок:

```


$$\int_0^2 (x^2 + 1) dx = \left[ \frac{x^3}{3} + x \right]_0^2$$


```

```
$$  
= \left( \frac{8}{3} + 2 \right) - \left( 0 + 0 \right) = \frac{8}{3} + 2 = \frac{14}{3}  
$$
```

****Відповідь:**** $\frac{14}{3} \approx 4.67$

Програмування

Блоки коду

Використовуй потрібні зворотні лапки з вказанням мови:

```
```python  
Ваш код тут
```
```

Python

```
```python  
Завдання: Знайти факторіал числа

def factorial(n):
 """
 Обчислює факторіал числа n.

 Args:
 n: невід'ємне ціле число

 Returns:
 факторіал числа n
 """
 if n < 0:
 raise ValueError("Факторіал визначений тільки для невід'ємних чисел")
 if n == 0 or n == 1:
 return 1
 return n * factorial(n - 1)

Тестування
if __name__ == "__main__":
 test_cases = [0, 1, 5, 10]
 for num in test_cases:
 print(f"{num}! = {factorial(num)}")
...

Результат виконання:
...
0! = 1
1! = 1
5! = 120
10! = 3628800
...`
```

---

### JavaScript

```
````javascript
/**
 * Сортування бульбашкою
 * @param {number[]} arr - масив чисел
 * @returns {number[]} - відсортований масив
 */
function bubbleSort(arr) {
    const n = arr.length;
    const result = [...arr]; // Копія масиву

    for (let i = 0; i < n - 1; i++) {
        for (let j = 0; j < n - i - 1; j++) {
            if (result[j] > result[j + 1]) {
                // Обмін елементів
                [result[j], result[j + 1]] = [result[j + 1], result[j]];
            }
        }
    }

    return result;
}

// Приклад використання
const numbers = [64, 34, 25, 12, 22, 11, 90];
console.log("До сортування:", numbers);
console.log("Після сортування:", bubbleSort(numbers));
````
```

---

## Java

```
````java
/**
 * Клас для роботи з бінарним деревом пошуку
 */
public class BinarySearchTree {
    private Node root;

    private class Node {
        int value;
        Node left, right;

        Node(int value) {
            this.value = value;
            left = right = null;
        }
    }

    /**
     * Вставка нового значення в дерево
     */
    public void insert(int value) {
        root = insertRec(root, value);
    }

    private Node insertRec(Node root, int value) {
        if (root == null) {
            return new Node(value);
        }
    }
}
```

```

        if (value < root.value) {
            root.left = insertRec(root.left, value);
        } else if (value > root.value) {
            root.right = insertRec(root.right, value);
        }

        return root;
    }

    /**
     * Пошук значення в дереві
     */
    public boolean search(int value) {
        return searchRec(root, value);
    }

    private boolean searchRec(Node root, int value) {
        if (root == null) return false;
        if (root.value == value) return true;

        if (value < root.value) {
            return searchRec(root.left, value);
        }
        return searchRec(root.right, value);
    }

    public static void main(String[] args) {
        BinarySearchTree tree = new BinarySearchTree();
        tree.insert(50);
        tree.insert(30);
        tree.insert(70);
        tree.insert(20);
        tree.insert(40);

        System.out.println("Пошук 40: " + tree.search(40)); // true
        System.out.println("Пошук 100: " + tree.search(100)); // false
    }
}
...

```

C++

```

...`cpp
#include <iostream>
#include <vector>
#include <algorithm>

/**
 * Швидке сортування (Quick Sort)
 */
class QuickSort {
public:
    static void sort(std::vector<int>& arr) {
        quickSort(arr, 0, arr.size() - 1);
    }

private:

```

```

static void quickSort(std::vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

static int partition(std::vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]);
    return i + 1;
}

};

int main() {
    std::vector<int> arr = {10, 7, 8, 9, 1, 5};

    std::cout << "До сортування: ";
    for (int x : arr) std::cout << x << " ";
    std::cout << std::endl;

    QuickSort::sort(arr);

    std::cout << "Після сортування: ";
    for (int x : arr) std::cout << x << " ";
    std::cout << std::endl;

    return 0;
}
...

```

C

```

````csharp
using System;
using System.Collections.Generic;

/// <summary>
/// Реалізація стеку на основі зв'язного списку
/// </summary>
public class LinkedStack<T>
{
 private class Node
 {
 public T Data { get; set; }
 public Node Next { get; set; }

 public Node(T data)
 {

```

```

 Data = data;
 Next = null;
 }
}

private Node top;
private int count;

public int Count => count;
public bool IsEmpty => count == 0;

public void Push(T item)
{
 Node newNode = new Node(item);
 newNode.Next = top;
 top = newNode;
 count++;
}

public T Pop()
{
 if (IsEmpty)
 throw new InvalidOperationException("Стек порожній");

 T data = top.Data;
 top = top.Next;
 count--;
 return data;
}

public T Peek()
{
 if (IsEmpty)
 throw new InvalidOperationException("Стек порожній");
 return top.Data;
}
}

class Program
{
 static void Main()
 {
 var stack = new LinkedStack<int>();

 stack.Push(10);
 stack.Push(20);
 stack.Push(30);

 Console.WriteLine($"Верхній елемент: {stack.Peek()}"); // 30
 Console.WriteLine($"Вилучено: {stack.Pop()}"); // 30
 Console.WriteLine($"Розмір стеку: {stack.Count}"); // 2
 }
}

```

---

## SQL

```
`` `sql
-- Створення таблиці студентів
CREATE TABLE students (
 id INT PRIMARY KEY AUTO_INCREMENT,
 first_name VARCHAR(50) NOT NULL,
 last_name VARCHAR(50) NOT NULL,
 email VARCHAR(100) UNIQUE,
 group_name VARCHAR(20),
 enrollment_date DATE,
 gpa DECIMAL(3,2)
);

-- Вставка даних
INSERT INTO students (first_name, last_name, email, group_name, enrollment_date, gpa)
VALUES
 ('Іван', 'Петренко', 'ivan@example.com', 'КН-21', '2021-09-01', 4.5),
 ('Марія', 'Коваленко', 'maria@example.com', 'КН-21', '2021-09-01', 4.8),
 ('Олексій', 'Шевченко', 'alex@example.com', 'КН-22', '2022-09-01', 4.2);

-- Вибірка студентів з GPA > 4.5
SELECT first_name, last_name, gpa
FROM students
WHERE gpa > 4.5
ORDER BY gpa DESC;

-- Середній GPA по групах
SELECT group_name, AVG(gpa) as avg_gpa
FROM students
GROUP BY group_name
HAVING AVG(gpa) > 4.0;
`` `
```



## Шаблон лабораторної роботи з програмування

# Лабораторна робота №1

## Тема: Алгоритми сортування

\*\*Студент:\*\* Іванов Іван Іванович

\*\*Група:\*\* КН-21

\*\*Варіант:\*\* 5

\*\*Дата:\*\* 06.02.2026

---

## 1. Мета роботи

Вивчити та реалізувати алгоритми сортування масивів.

---

## 2. Завдання

Реалізувати алгоритм сортування злиттям (Merge Sort) та проаналізувати його складність.

---

## 3. Теоретичні відомості

Сортування злиттям – ефективний алгоритм сортування зі складністю  $O(n \log n)$ .

Алгоритм базується на принципі "розділяй і володарюй":

1. Розділити масив на дві половини
2. Рекурсивно відсортувати кожену половину
3. Злити відсортовані половини

---

#### ## 4. Реалізація

```
```python
def merge_sort(arr):
    """
    Сортування злиттям.

    Часова складність:  $O(n \log n)$ 
    Просторова складність:  $O(n)$ 
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    """Злиття двох відсортованих масивів."""
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
```
```

---

#### ## 5. Тестування

```
```python
import random
import time

# Тест 1: Базовий тест
arr1 = [38, 27, 43, 3, 9, 82, 10]
print(f"Вхід: {arr1}")
print(f"Вихід: {merge_sort(arr1)}")

# Тест 2: Великий масив
```

```
arr2 = [random.randint(1, 1000) for _ in range(10000)]
start = time.time()
sorted_arr = merge_sort(arr2)
end = time.time()
print(f"Час сортування 10000 елементів: {end - start:.4f} сек")
```

```

```
Результати:
```

```

```
Вхід: [38, 27, 43, 3, 9, 82, 10]
Вихід: [3, 9, 10, 27, 38, 43, 82]
Час сортування 10000 елементів: 0.0523 сек
```

```

## ## 6. Аналіз складності

| Параметр          | Значення      |
|-------------------|---------------|
| Найкращий випадок | $O(n \log n)$ |
| Середній випадок  | $O(n \log n)$ |
| Найгірший випадок | $O(n \log n)$ |
| Додаткова пам'ять | $O(n)$        |

## ## 7. Висновки

В ході лабораторної роботи було реалізовано алгоритм сортування злиттям. Алгоритм показав стабільну продуктивність  $O(n \log n)$  незалежно від вхідних даних.

## Конвертація в PDF

### VS Code + Markdown Preview Enhanced

1. Відкрий .md файл
2. Натисни **Ctrl+Shift+V** для попереднього перегляду
3. Клікни правою кнопкою → **Export** → **PDF**

### Онлайн конвертери

- [md2pdf.netlify.app](https://md2pdf.netlify.app)
- [dillinger.io](https://dillinger.io) (Export as PDF)
- [pandoc.org](https://pandoc.org) (командний рядок)

### Pandoc (командний рядок)

```
pandoc input.md -o output.pdf --pdf-engine=xelatex
```

## Чек-лист перед здачею

- ☐ Вказано ПІБ, групу, дату

- ☐ Код відформатований і має коментарі
- ☐ Формули правильно відображаються
- ☐ Є результати виконання/тестування
- ☐ Файл конвертовано в PDF
- ☐ Розмір файлу до 10 МБ